

Avance del Proyecto — Fase II

Sistema de Avisos para un Gimnasio (Versión Final)

1. Análisis técnico

Análisis organizacional

La organización que **elegí** para este proyecto es un gimnasio. El área en la que se enfoca mi proyecto es la **operación, mantenimiento de las instalaciones y control de equipo**.

En la cotidianidad de un gimnasio ocurren diversas incidencias, tales como máquinas de ejercicio dañadas, áreas comunes sucias, desabasto o fallas en el suministro de agua, o material de entrenamiento deteriorado. Estos eventos requieren una atención oportuna y un seguimiento transparente para determinar si el problema se encuentra en estado **Pendiente, En proceso o Resuelto**.

Para dar respuesta a esta necesidad, **diseñé y consolidé** la versión final de un **Sistema de Avisos y Registro de Incidencias en Consola**. Esta versión incorpora lectura/escritura permanente en archivos de texto externos (.txt), control dinámico de inactividad del usuario, validaciones estrictas de datos de entrada (fechas y tiempos), manejo robusto de excepciones de sistema y depuración técnica mediante el módulo estándar PDB de Python.

2. Definición del problema

La problemática principal que **identifiqué** radica en la falta de una herramienta centralizada y automatizada para reportar, almacenar y consultar los problemas operativos de un gimnasio.

Mi programa permite seleccionar la categoría de la falla, ingresar descripciones personalizadas, actualizar el estado de resolución y registrar los tiempos estimados. La información no se pierde al cerrar la aplicación, ya que se almacena de forma persistente en archivos de texto independientes con sello de fecha estructurada.

Reglas de negocio

1. El programa debe solicitar el nombre del usuario y dar una bienvenida dinámica.
2. Debe mostrar una pantalla de carga dinámica tipo barra de progreso (máximo 5 segundos).

3. Debe registrar la fecha de operación mediante una captura validada en formato de día ($1 \leq \text{día} \leq 31$), mes ($1 \leq \text{mes} \leq 12$) y año ($2000 \leq \text{año} \leq 2100$), almacenándola en la tupla `Fecha = (día, mes, año)`.
4. El menú principal debe presentarse formateado en estructura de **matriz** y ser controlado por un bucle `while`.
5. Debe incluir un control de inactividad mediante un ciclo `for`: si transcurren **10 minutos (600 segundos)** sin interacción, el programa se pausa y pregunta si el usuario desea continuar ("si") o regresar al inicio ("no").
6. Las opciones de estado disponibles para cada aviso son: **Pendiente, En proceso y Resuelto**.
7. Si el estado seleccionado es **Resuelto**, el tiempo estimado de solución se fija automáticamente en 0 minutos. Para otros estados, no se permiten tiempos negativos ($t \geq 0$).
8. El sistema debe contar con **4 archivos .txt de prueba preexistentes** (`avisos_maquinas.txt`, `avisos_limpieza.txt`, `avisos_agua.txt`, `avisos_material.txt`).
9. Cada reporte escrito o nota adicional debe anexar obligatoriamente la fecha guardada en la tupla.
10. Se deben capturar posibles errores de tiempo de ejecución (`ValueError`, `FileNotFoundError`, `PermissionError`) para evitar cierres abruptos.

3. Listado de requerimientos funcionales

1. **Autenticación e Interfaz de Carga:** Solicitar el nickname del usuario, desplegar la bienvenida concatenada y ejecutar la barra de carga antes del menú.
2. **Validación de Fecha:** Capturar y empaquetar la fecha en una tupla estructurada antes de acceder a la gestión de archivos.
3. **Despliegue del Menú Matricial:** Mostrar las opciones de registrar aviso, leer archivos, agregar información y salir en formato de matriz de dos dimensiones.
4. **Medición de Inactividad:** Monitorear el tiempo sin respuesta del usuario mediante bucle `for` y evaluar la suspensión a los 10 minutos.
5. **Registro de Incidencias:** Generar avisos específicos según la categoría elegida (o descripción libre) y asignar tiempo/estado sin permitir valores negativos.
6. **Persistencia (Escritura / Anexado):** Abrir el archivo `.txt` correspondiente en modo adición ("a") e integrar la fecha en la tupla.
7. **Lectura de Archivos:** Desplegar los archivos de texto disponibles en lista numerada y mostrar su contenido en consola utilizando modo lectura ("r").
8. **Manejo de Excepciones:** Interceptar entradas inválidas o fallos de E/S con bloques `try-except`.
9. **Módulo de Depuración (PDB):** Permitir la ejecución de puntos de interrupción técnicos cuando la variable `modo_debug` esté activa.

4. Clasificación de datos

Dato	Tipo	Uso / Descripción
nombre	Cadena (<i>string</i>)	Guardar el nickname o nombre introducido por el usuario.
Fecha	Tupla	Almacenar de forma inmutable la fecha estructurada: (dia, mes, año).
dia, mes, año	Entero (<i>int</i>)	Variables temporales para la validación de rangos de fecha.
opcion / opcion_menu	Entero (<i>int</i>)	Guardar la opción seleccionada dentro de los menús.
problema	Cadena (<i>string</i>)	Guardar la categoría o descripción del problema reportado.
estado	Cadena (<i>string</i>)	Indicar el estado de resolución (<i>Pendiente, En proceso, Resuelto</i>).
tiempo_solucion	Entero (<i>int</i>)	Registrar el tiempo estimado en minutos (debe ser ≥ 0).
archivos	Lista (<i>list</i>)	Almacenar la lista de nombres de los 4 archivos .txt disponibles.
menu	Matriz (<i>list of lists</i>)	Representar la estructura bidimensional del menú principal.
modo_debug	Booleano (<i>bool</i>)	Activar o desactivar las trazas de depuración con PDB.

5. Operadores del lenguaje

- **Asignación (=):** Guardado de datos en variables y empaquetado de la tupla Fecha.
- **Comparación (==, !=, <, >, <=, >=):** Evaluación de opciones del menú, rangos de días/meses/años y validación de tiempos no negativos.
- **Aritméticos (+, *, -):** Cálculo de porcentajes para la pantalla de carga, construcción de barras con cadenas ("" * i) y resta de tiempo en el control de inactividad.
- **Lógicos (and, or, not):** Combinación de condiciones para validar rangos numéricos y controlar los estados de los bucles.
- **Concatenación de Cadenas (+ / f-strings):** Creación del mensaje de bienvenida, formateo de rutas y escritura de líneas en archivos de texto.

6. Estructuras de control

- **if / elif / else:** Utilizados para la toma de decisiones al categorizar fallas, asignar archivos destino, evaluar estados y validar entradas de fecha.
- **while:** Mantiene la ejecución continua del sistema principal y la reevaluación de menús hasta que el usuario decida salir explícitamente.
- **for:** Empleado en el despliegue del menú en forma de matriz, en el conteo iterativo de la pantalla de carga, en la lectura/listado de archivos y en la simulación del temporizador de inactividad de 10 minutos.

- **try / except:** Controlan excepciones como ValueError (ingreso de letras por números), FileNotFoundError (archivos no encontrados) y PermissionError (bloqueo de archivos).

7. Modelo Entrada - Proceso - Salida (EPS)

Entrada

1. Nickname del usuario.
2. Día, mes y año de operación.
3. Opción del menú principal seleccionada desde la matriz.
4. Selección de la categoría de problema (o descripción manual).
5. Selección del estado de la incidencia.
6. Tiempo estimado en minutos (si no está resuelto).
7. Contenido o notas adicionales a anexar en archivos .txt.

Proceso

1. Validar e imprimir el mensaje de bienvenida y ejecutar la barra de carga dinámica.
2. Validar que la fecha ingresada sea coherente y construir la tupla Fecha = (día, mes, año).
3. Presentar el menú estructurado en matriz y verificar el temporizador de inactividad de 600 segundos en el ciclo for.
4. Procesar la opción elegida mediante bloques try-except.
5. Si se registra un aviso, calcular el tiempo, determinar el archivo .txt correspondiente y anexar la información con saltos de línea claros (\n) y fecha formateada.
6. Si se solicita lectura, abrir el archivo .txt elegido y transmitir su contenido a pantalla.

Salida

1. Pantalla de carga animada con porcentaje visual.
2. Confirmación del registro e impresión del reporte en consola.
3. Contenido desplegado de los archivos .txt leídos.
4. Actualización física permanente de los archivos .txt de prueba.
5. Avisos de error amigables en caso de fallas de entrada o de lectura/escritura de archivos.

8. Pseudocódigo (Actualizado a Fase II)

Proceso Sistema_Avisos_Gimnasio_Fase2

Definir nombre, problema, estado, archivo, informacion Como Caracter

Definir día, mes, año, opción, estado_opcion, tiempo, opcion_menu Como Entero

Definir Fecha Como Tupla

Definir archivos Como Lista de Cadenas

```
archivos = ["avisos_maquinas.txt", "avisos_limpieza.txt", "avisos_agua.txt", "avisos_material.txt"]
```

```
Escribir "Ingresa tu nombre o nickname:"
```

```
Leer nombre
```

```
Escribir "Bienvenido/a " + nombre + " al Sistema de Avisos del Gimnasio!"
```

```
// Pantalla de carga
```

```
Para i = 0 Hasta 10 Con Paso 1 Hacer
```

```
    Escribir "[Barra de progreso]", (i * 10), "%"
```

```
    Pausa(0.15 segundos)
```

```
FinPara
```

```
// Captura y validación de fecha
```

```
Repetir
```

```
    Escribir "Ingresa el día (1-31):"
```

```
    Leer día
```

```
Hasta Que día >= 1 Y día <= 31
```

```
Repetir
```

```
    Escribir "Ingresa el mes (1-12):"
```

```
    Leer mes
```

```
Hasta Que mes >= 1 Y mes <= 12
```

```
Repetir
```

```
    Escribir "Ingresa el año (2000-2100):"
```

```
    Leer anio
```

```
Hasta Que anio >= 2000 Y anio <= 2100
```

```
Fecha = (día, mes, anio)
```

```
Repetir
```

```
    Escribir "--- Menú Principal ---"
```

```
    Escribir "1. Registrar aviso"
```

```
    Escribir "2. Leer archivos"
```

```
    Escribir "3. Agregar información"
```

```
    Escribir "4. Salir"
```

```
// Control de inactividad (600 segundos = 10 minutos)
```

```
Para segundo = 0 Hasta 600 Hacer
```

```
    Si Inactividad >= 600 Entonces
```

```
        Escribir "Han transcurrido 10 minutos de inactividad. ¿Deseas continuar? (si/no)"
```

```
        Leer respuesta_inactiva
```

```
        Si respuesta_inactiva != "si" Entonces
```

```
            Salir del Bucle Menú
```

```
        FinSi
```

```
    FinSi
```

```
FinPara
```

```
Leer opcion_menu
```

```
Segun opcion_menu Hacer
```

```
    1:
```

```
        Escribir "Selecciona problema: 1. Máquina, 2. Limpieza, 3. Agua, 4. Material, 5. Otro"
```

```
        Leer opcion
```

```

Escribir "Selecciona estado: 1. Pendiente, 2. En proceso, 3. Resuelto"
Leer estado_opcion

Si estado_opcion == 3 Entonces
    tiempo = 0
Sino
    Repetir
        Escribir "Tiempo estimado en minutos:"
        Leer tiempo
    Hasta Que tiempo >= 0
FinSi

// Persistencia en archivo .txt
AbrirArchivoModoAppend(archivo)
EscribirLinea("=== AVISO DEL GIMNASIO ===")
EscribirLinea("Problema: " + problema)
EscribirLinea("Estado: " + estado)
EscribirLinea("Tiempo: " + tiempo + " minutos")
EscribirLinea("Fecha: " + Fecha)
CerrarArchivo()

2:
MostrarListaArchivos(archivos)
Leer archivo_seleccionado
AbrirYLeerArchivo(archivo_seleccionado)

3:
MostrarListaArchivos(archivos)
Leer archivo_seleccionado
Escribir "Escribe la información a agregar:"
Leer informacion
AnexarInformacion(archivo_seleccionado, informacion, Fecha)

4:
Escribir "Sistema finalizado."
FinSegun

Hasta Que opcion_menu == 4

FinProceso

```

9. Diagrama de Flujo del Proceso Integrado

